

Scalable Network I/O

From `select()` to `io_uring`: Benchmarking Five I/O Engines

Single-threaded TCP echo servers: a deep dive into Linux socket mechanisms

select



poll



epoll



io_uring

Project Statement & Objectives

THE PROBLEM

Modern servers must juggle thousands of simultaneous TCP connections on a single thread. The I/O multiplexing mechanism chosen decides throughput, latency, memory footprint, and CPU cost.

Mechanism	Year	Core Design
select	1983	Bitmap of fds, full copy per call; hard 1024-fd kernel limit
poll	1997	Array of structs, no fd limit; full array copy per call
epoll	2002	Kernel-resident interest list, $O(1)$ event delivery
io_uring	2019	Async ring buffers, batched syscalls via shared mmap

OBJECTIVES

- 1 Implement : 4 single-threaded C servers; one per mechanism
- 2 Benchmark : Concurrent loads from 10 → 10,000 using tcpkali
- 3 Measure : Throughput, p99 latency, syscalls, context switches
- 4 Analyse : Theory vs. reality across all four metrics
- 5 Demonstrate : The newest mechanism is not always the fastest

Design & Architecture

Five Engines, One Shared Interface

Shared foundation (`network_utils.c`) : socket creation, non-blocking mode (`fcntl`), `TCP_NODELAY`, `SO_REUSEADDR`
Every server : single-threaded, port 8080, echoes bytes back with a 64KB per-client output buffer and flush-until EAGAIN state machine

1

1983

select

- Custom `my_fd_set` bitmap, cast to `fd_set*`; kernel scans and copies back; user scans all bits. Hard kernel limit: 1024 fds.
- Complexity / Trait: $O(N)$ scan, capped at 1024 clients

2

1997

poll

- Passes a `pollfd` array on every call; kernel fills revents; user walks the whole array. No fd-count ceiling — capped at 10,000 for fairness.
- Complexity / Trait: $O(N)$ copy, up to 10,000 clients

3

2002

epoll

- Interest list lives in the kernel as a red-black tree; `epoll_wait` returns only ready fds. State transitions managed via `epoll_ctl(MOD)` to switch between `EPOLLIN` / `EPOLLIN|EPOLLOUT`.
- Complexity / Trait: $O(1)$ event delivery

4

2019

io_uring

- Submit ops via an SQ ring; kernel completes async, posts to a CQ ring via shared mmap. `SOCK_NONBLOCK` set directly in the ACCEPT SQE; no `fcntl()` round-trip per connection. `MAX_CLIENTS` (10,000) is actively enforced via an `active_clients` counter.
- Complexity / Trait: Async rings, batched `io_uring_enter` calls

Fixes & Benchmark Results

1 Enforcing select's 1024-fd Limit

Custom my_fd_set bitmap sized to MAX_CLIENTS = 1024. Connections with fd ≥ 1024 are rejected cleanly. select is tested only up to 1000 connections where it can operate correctly.

2 Idiomatic io_uring Accept (fcntl Removed):

accept_flags = SOCK_NONBLOCK in the ACCEPT SQE hands back an already-nonblocking fd directly from the kernel; no fcntl(F_GETFL)/fcntl(F_SETFL) round-trip per connection.

3 MAX_CLIENTS Actually Enforced for io_uring:

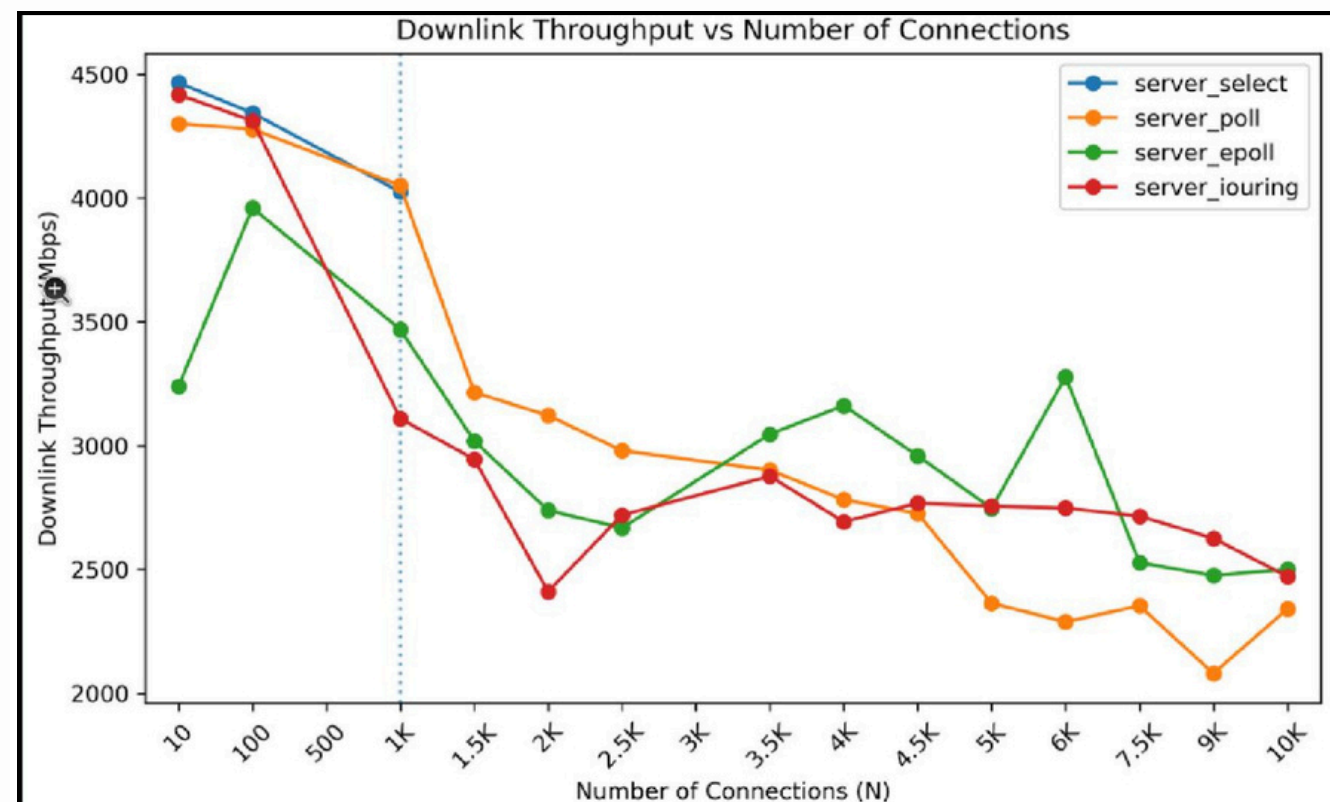
Added an active_clients counter (incremented in alloc_client, decremented in free_client). Connections past 10,000 are rejected via submit_close.

4 Output Buffer + Flush State Machine (all engines):

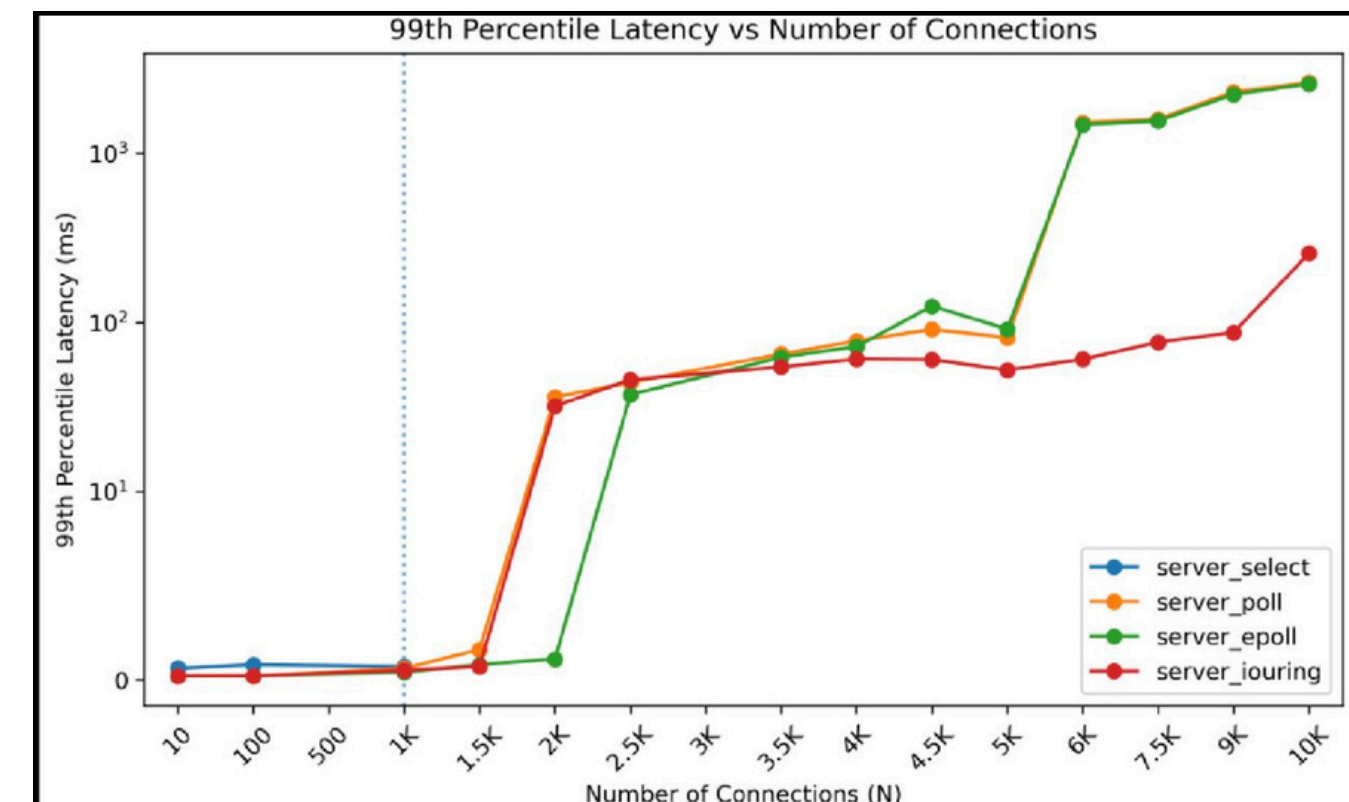
64 KB per-client buffer; flush_outbound_buffer retries until EAGAIN; memmove shifts remaining bytes; zero data loss under backpressure.

5 Mid-loop SQ Drain (maybe_flush):

If fewer than 4 SQ slots remain during CQ drain, ring_submit(r, 0) is called immediately to prevent ring exhaustion under 5000+ connection bursts.



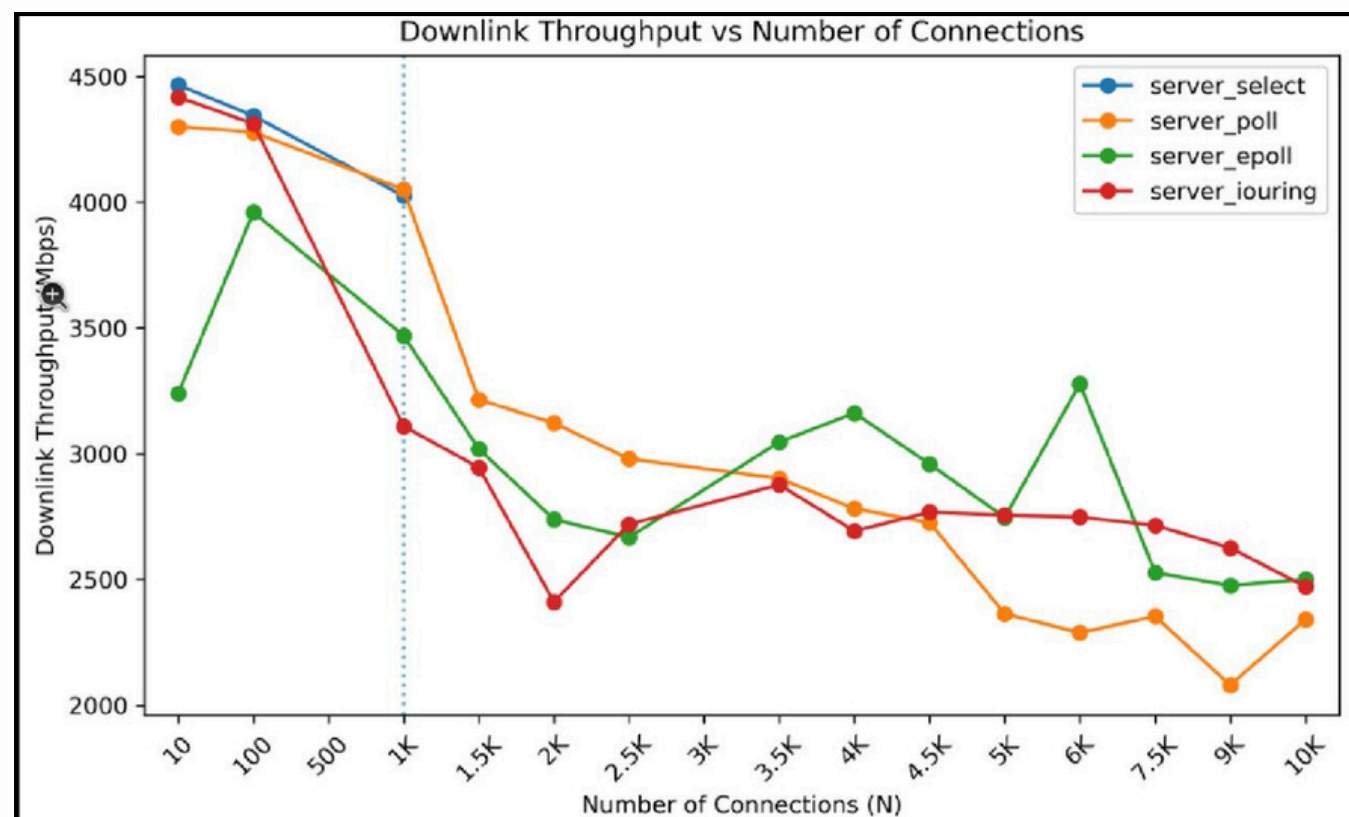
- select: ~4025 Mbps (fails at 1500+ conns)
- poll: ~4050 Mbps, degrades to ~2364 Mbps at 5000 conns
- epoll: ~3469 Mbps at 1000, ~2747 Mbps at 5000 conns
- io_uring: ~3109 Mbps at 1000, holds ~2756 Mbps at 5000 conns



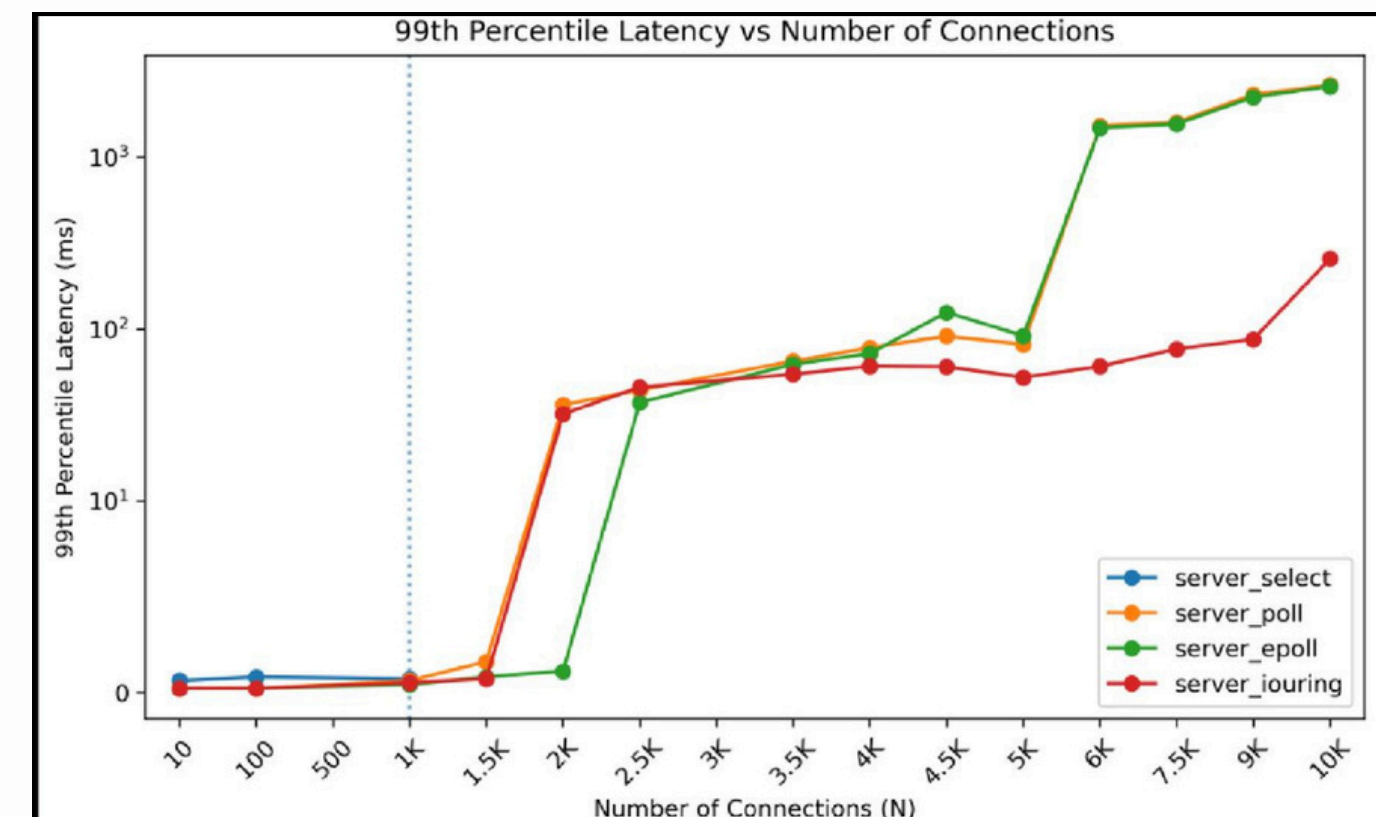
- At 1000 connections, all four engines: 0.4–0.7 ms
- At 5000 conns — poll: 81.2 ms, epoll: 91.3 ms, io_uring: 52.3 ms

Non-functional Analysis

Setup : tcpkali (core 1) drives load against the server (core 0) · loads swept 10 → 10,000 connections · measured over a 10s sustained window · 512-byte message payload · connect-rate 10,000/s

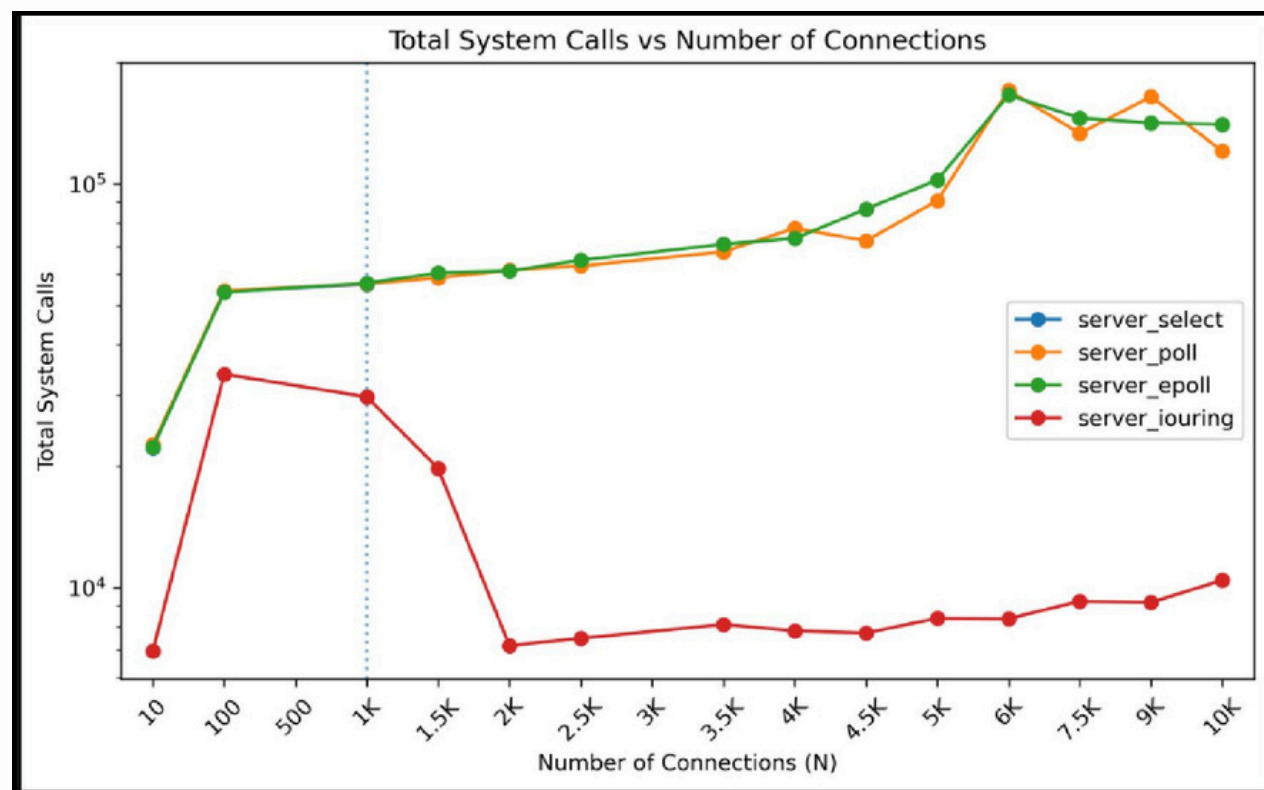


- select: Competitive at low load (~4465 Mbps @ 10 conns, ~4025 Mbps @ 1000 conns), then collapses to NaN at 1500+ conns due to the 1024-fd hard limit.
- poll: Starts strong (~4300 Mbps @ 10 conns), degrades steadily to ~2364 Mbps @ 5000 conns due to $O(N)O(N)$ full-array copy on every call.
- epoll: ~3240 Mbps @ 10 conns, peaks ~3960 Mbps @ 100 conns, holds ~2747 Mbps @ 5000 conns. Most consistent curve at mid-to-high load.
- io_uring: ~4417 Mbps @ 10 conns, drops to ~3109 Mbps @ 1000 conns as CQE harvest cost grows; stabilises ~2472–2756 Mbps at 5000–10,000 conns.



- All engines track closely below 1000 conns (< 1 ms p99).
- At 2000 conns: poll jumps to 36.2 ms, epoll to 37.4 ms, io_uring to 32.0 ms.
- At 5000 conns: poll 81.2 ms, epoll 91.3 ms, io_uring 52.3 ms.
- At 6000+ conns: poll and epoll spike to 1.5–2.6 s; io_uring stays in the 52–88 ms range.
- select shows sub-millisecond p99 at ≤ 1000 conns (its valid operating range).

Non-functional Analysis



- select / poll / epoll: Dominated by sendto + recvfrom at low loads (10K–26K calls @ 10 conns); primary multiplexing syscall (pselect6 / poll / epoll_wait) is comparatively rare.
- io_uring: sendto/recvfrom replaced by ring operations; io_uring_enter calls reduced via batching — measurably fewer multiplexing-layer syscalls at scale.
- Paradox: Fewer syscalls ≠ fastest — memory-barrier cost on the SQ/CQ rings and CQE harvest overhead replaces syscall overhead for small 512-byte echo messages.

RELIABILITY & CORRECTNESS

Graceful disconnect

EAGAIN handled

SIGPIPE suppressed

Ring-exhaustion guard

Zero data loss

Challenges & Key Takeaways

1

select's 1024-fd Limit

- Problem: The Linux kernel's `select()` syscall silently ignores `nfds > 1024` regardless of bitmap size — memory corruption risk above fd 1024.
- Fix: `MAX_CLIENTS = 1024`; connections with `fd ≥ 1024` rejected cleanly; `select` benchmarked only at `≤ 1000` conns where it is correct.

2

Partial Sends

- Problem: `send()` silently drops data when the kernel send buffer fills under load.
- Fix: Per-client 64 KB output buffer; `flush_outbound_buffer` loops until `EAGAIN`; buffer compacted with `memmove`.

3

Ring Exhaustion (io_uring)

- Problem: `io_uring` SQ fills up during burst accept + `recv/send` at 5000+ connections.
- Fix: `maybe_flush()` drains the SQ mid-loop whenever fewer than 4 slots remain.

4

io_uring fcntl Overhead

- Problem: Earlier build added `fcntl(F_GETFL)/fcntl(F_SETFL)` per accepted connection, undermining the zero-syscall premise.
- Fix: `accept_flags = SOCK_NONBLOCK` in the `ACCEPT` SQE; kernel delivers an already-nonblocking fd — no round-trip.

5

Strace Interference

- Problem: `ptrace` overhead skewed syscall counts 10×.
- Fix: `strace` attached post ramp-up; measurements taken over a clean 10 s window.

KEY TAKEAWAYS

Does $O(N)$ hurt <code>select/poll</code> ?	<code>select</code> hits a hard 1024-fd wall; valid only under 1000 conns. <code>poll</code> degrades gracefully but latency explodes past 2000 conns.
Is <code>epoll</code> truly $O(1)$?	For event notification, yes. But at 6000+ conns the kernel TCP stack — not <code>epoll</code> — saturates, causing latency spikes to 1–2 s.
Does <code>io_uring</code> cut syscall overhead?	Yes, <code>io_uring_enter</code> calls are batched and <code>sendto/recvfrom</code> replaced by ring ops. But memory-barrier cost on small 512-byte messages limits the gain.
Is newer always faster?	No. At moderate loads (100–1000 conns), <code>poll</code> and <code>select</code> match or beat <code>epoll</code> on throughput. <code>io_uring</code> shows the best p99 latency at 5000–10,000 conns but not the highest throughput.
When does <code>io_uring</code> make sense?	Large I/O payloads, CPU-pinned always-busy workloads, or when minimising syscall count matters more than raw echo throughput on tiny messages.